

面向版本化规范文档的证据约束神经符号本体语义漂移修复方法

Anonymous Author(s)

Abstract

版本化规范文档持续修订定义、义务、例外、适用范围和生效条件，使基于旧版本规范构建的OWL本体产生当前证据不再支持的语义失配。本文提出ECR-REPAIR，一种以Semantic IR为neural-to-symbolic interface的证据约束模块化神经符号本体修复框架：LLM负责候选内容隔离的规范证据解释，确定性恢复与约束感知排序负责符号语义决策，仅在排序歧义时触发候选级神经蕴含验证，最终由OWL/RDF materialization、Reasoner、graph-delta与冻结repair/CQ regression checks执行形式安全验证。本文评价条件式、evidence-relative semantic drift repair，即潜在事件已定位、相关evidence window已提供且有限候选中包含正确修复，而不把上游漂移发现、开放语料召回或开放式候选生成计入主指标。在方法冻结后构建的RFC blind-large benchmark上，264个事件、1320次调用取得1319/1320 (99.92%) closure accuracy和263/264 (99.62%) strict event success，且wrong-selected rate为0%。进一步的人工temporal semantic audit、formal-closure stress test、候选池扩容、v5跨后端复现和full-online candidate-identifier permutation audit分别检验了语义表示、形式安全与鲁棒性边界。结果表明，高可靠本体修复的关键不仅在于提高LLM输出正确率，更在于建立显式、可审计、可弃权且责任边界清晰的神经-符号接口。

Keywords

ontology evolution, semantic drift, neuro-symbolic ontology repair, Semantic IR, large language models, OWL, selective prediction

1 引言

本体常用于表示标准规范、监管规则、协议约束和领域知识。当其知识来源是规范文档时，本体的正确性不仅取决于逻辑一致性，还取决于其是否仍然被当前版本的规范证据支持。现实中，规范文档会不断更新：新版本可能替代旧版本，例外条款可能覆盖一般规则，限定语可能改变多个句子的作用域，或者某个属性值可能因为修订而失效。这些变化会导致本体出现语义漂移。

语义漂移不同于普通语法错误。本体可能仍然可以被reasoner正常加载，也没有明显的逻辑不一致，但其中某个属性值、规则约束或适用范围已经不再符合当前规范。对于这类问题，只做文本差异比较不够，只做OWL consistency checking也不够。系统需要理解规范文本中的版本关系、例外结构、作用域关系，并把这种理解转化为可验证的本体修复操作。

本文对“语义漂移”采用分层的操作性定义。令

$$\text{Support}(a, D_t) \in \{\text{supported}, \text{contradicted}, \text{underspecified}\}$$

表示断言 a 相对于时刻/版本 t 的规范证据状态。若保留了完整历史证据，则严格的版本漂移要求存在 t_0, t_1 和替代断言 a' ，满足

$$t_0 < t_1, \quad a' \neq a,$$

$$\text{Support}(a, D_{t_0}) = \text{supported}, \quad \text{Support}(a, D_{t_1}) \neq \text{supported},$$

$$\text{Support}(a', D_{t_1}) = \text{supported}.$$

但v5并不要求每个事件都保存可独立验证的 $t_0 \rightarrow t_1$ 完整历史证据链，因此主实验实际评价更保守的**evidence-relative semantic drift repair**。令 D_{cur} 表示当前冻结证据，则目标断言 a_0 与正确修复 a^* 满足

$$\text{Support}(a_0, D_{\text{cur}}) \neq \text{supported},$$

$$\exists a^* \in C_E : \text{Support}(a^*, D_{\text{cur}}) = \text{supported}.$$

Temporal-Version事件通常直接编码版本/生效关系；General Rule-Exception / Cross-Sentence Scope也可能表现为当前证据下暴露出的规则或作用域语义失配。后续严格provenance audit在不放宽lineage与support条件的情况下得到 $N = 0$ ，因此本文明确不把主benchmark表述为provenance-complete historical drift benchmark。

大语言模型为规范文本理解提供了新的可能，但直接使用LLM也带来风险。模型可能输出格式正确但语义失真的JSON；可能因为候选描述写得更清楚而选择某个候选；可能受到候选顺序或编号的影响；也可能在证据不足时仍然给出确定修复。因此，本文不把问题简化为“让LLM直接选择答案”，而是将其拆解为证据理解、语义中间表示、候选映射、选择性门控和形式化验证等多个可审计阶段。

ECR-REPAIR将该任务建模为模块化神经-符号修复过程，而不是单一LLM决策。神经语义模块负责从自然语言规范证据中形成可追溯的语义解释；符号语义模块将解释约束为类型化Semantic IR、确定性时间关系和可检查的候选评分；只有当符号排序无法唯一决策时，候选级LLM蕴含验证才作为fallback重新介入；最终由OWL/RDF patch、Reasoner、图差分和冻结repair/CQ regression checks完成形式符号验证。Semantic IR因而承担neural-to-symbolic interface（神经到符号语义接口）的角色。

与此同时，ECR-REPAIR采用候选内容隔离（**candidate-content isolation**）：候选value、operation、id、score与top-k identity均不得反馈到语义抽取/恢复模块。排序失败最多只返回不携带候选语义的ranking-failure flag，并触发一次内容隔离恢复。该设计使神经模块负责“解释证据”，符号模块负责“约束决策与验证”，避免把LLM的自由生成直接等价为本体修改。

本文研究问题可以表述为：

如何构建一个证据约束、候选内容隔离、可弃权且可形式验证的神经符号修复框架，将版本化规范文档中的自然语言证据可靠地转化为OWL/RDF本体修复决策？

本文主要贡献如下：

- 提出面向版本化规范文档的本体语义漂移修复问题定义，将修复任务建模为证据约束的有限候选选择与OWL闭环验证问题。
- 提出ECR-REPAIR，一种候选内容隔离的模块化神经符号本体修复架构，以显式Semantic IR连接LLM的证据语义解释、符号语义约束、候选决策和OWL/RDF形式验证。
- 设计“神经语义解释→符号语义决策→必要时神经蕴含验证→形式符号闭环”的可诊断接口，并针对General Rule-Exception、Cross-Sentence Scope和Temporal-Version中的结构/时间歧义提供可审计恢复机制。

- 在方法冻结后构建并冻结一个更大规模RFC 真实规范文档blind-large benchmark, 覆盖264个事件、11个领域、13个实际入选RFC 来源族和三类语义漂移类型。
- 通过主实验、阶段消融、人工temporal semantic audit、形式闭环stress test、候选池扩容、候选identifier shortcut audit、v5 跨后端复现和benchmark provenance audit 系统验证方法效果, 并明确形式闭环、repair/CQ checks、候选构造与event-level benchmark 的评价边界。

2 相关工作

2.1 本体演化与本体修复

本体演化研究关注知识库随领域变化而变化时如何保持一致性、可维护性和语义正确性 [3, 6]。OWL 本体可以通过reasoner 检查逻辑一致性 [17], 但逻辑一致并不意味着语义仍然符合最新规范。已有本体修复研究多关注不一致公理删除、冲突定位、unsatisfiable concept repair 和ontology alignment repair [7, 8]。SHACL 及其repair 工作则从RDF constraint validation 和minimal repair 角度处理数据图约束违反 [1, 2, 9]。本文关注的问题不同: 源本体未必不一致, 但其断言相对于版本化规范文档已经过时。

2.2 RAG 与证据选择

检索增强生成通常通过外部证据约束模型回答 [10]。近期工作进一步强调reranker 不只追求相关性, 还应选择对生成器足够、可解且不过度泄漏的证据 [15]。本文的谓词感知证据聚焦将这一思想用于版本化规范文档中的本体修复。

2.3 法律与监管信息抽取

法律和监管文本中的信息抽取长期面临定义、义务、例外、条件、优先级和跨句作用域等结构性挑战 [13]。Legal AMR 研究表明法律语义结构化需要处理领域词汇、角色关系和复杂语义 [16]。监管规则抽取和法律需求翻译工作也探索了structured rule set、canonical representation 与executable code [5, 14]。本文的最终目标不是单独抽取规则, 而是将结构化语义用于有限候选修复和OWL 闭环验证。

2.4 大模型辅助本体工程

大模型辅助本体工程研究关注如何使用LLM 支持本体需求获取、概念生成、关系建模、本体评价和维护 [11]。近期ontology generation 工作进一步探索从user stories 和competency questions 生成OWL ontology draft 的可能性 [12]。这些工作说明神经语言模型能够降低自然语言到本体工程表示之间的转换成本, 但也带来hallucination、结构不稳定和缺少形式保证等风险。本文采用更保守的定位: LLM 不直接任意改写本体, 而是先形成evidence-grounded SEMANTIC IR, 再在显式候选决策和形式验证层中受约束地执行修复。

2.5 神经符号推理与本体修复

本文所称“神经符号”是模块化的neural-symbolic integration, 而不是端到端神经逻辑联合训练或可微定理证明。LLM 本体工程工作表明神经模型能够承担自然语言语义解释与结构生成 [11, 12], 而OWL 形式语义和reasoner 提供显式、可检查的符号约束 [17]。ECR-REPAIR以Semantic IR 作为二者之间的接口契约: 神经模块负责从规范文本中抽取和细化evidence-grounded semantics, 并仅在候选歧义时执行fallback entailment

verification; 符号模块负责类型约束、确定性temporal normalization、constraint-aware ranking, 以及OWL/RDF consistency、graph-delta 和repair/CQ regression validation。该模块化架构的目标不是让符号层事后“替LLM 兜底”, 而是让神经解释从一开始就通过显式中间表示进入可审计的符号决策链。

2.6 选择性预测与可信修复

选择性预测研究关注模型在低置信度情况下abstain, 从而控制覆盖率与风险之间的权衡 [4]。本文将这一思想引入本体修复: 证据、Semantic IR、候选决策或形式化验证不满足条件时, 系统fail-closed。

3 问题定义

给定版本化规范文档集合 D 、可能发生语义漂移的源本体 O_0 , 以及一个修复事件 E 。每个事件包含公开元数据, 包括subject、predicate、semantic type、case context 和证据来源。记 $D_E \subseteq D$ 为该事件已提供的证据窗口, M_E 为对应的公开事件元数据。对于每个事件, 系统给定有限候选集合

$$C_E = \{c_1, c_2, \dots, c_k\}.$$

每个候选包含display value 和形式化OWL/RDF 编辑操作。私有oracle 候选 c_E^* 只在离线评价阶段读取, 不能进入模型输入、语义抽取/恢复或候选决策的提示中。

条件式任务假设。本文研究的是conditional repair decision, 而不是完整端到端ontology maintenance。对每个正式事件, (i) 潜在漂移位置/目标subject-predicate 已给定; (ii) 与该事件相关的evidence source/window 已提供, EvidenceFocus 只在这些已提供证据上进行谓词感知裁剪与聚焦, 而不评价开放语料检索recall; (iii) 主v5 benchmark 的候选集合固定为3项, 并保证包含一个经人工复核的正确修复。因而本文不评价drift discovery、open-corpus evidence retrieval 或open-ended candidate generation。为检验有限候选规模敏感性, 额外构造保持parent benchmark 不变的 $k = 5$ 与 $k = 10$ hard-negative variants; 这些实验只支持有限候选池扩容的robustness 结论, 不代表开放式候选生成或任意大候选池。

方法输出可以是某个候选 $\hat{c}_E$, 也可以是ABSTAIN。若选择候选, 则只有同时满足以下条件才记为修复闭环成功:

- (1) 选择候选与私有oracle 一致;
- (2) 候选OWL patch 成功物化;
- (3) 修复后本体通过reasoner 一致性检查;
- (4) 实际RDF 图差分与候选声明的编辑一致, 不出现额外未声明triple;
- (5) 冻结的repair/CQ regression checks 通过。

这里不主张在所有可能修复中求解全局最小编辑。本文所谓“最小/受限修改”被操作性地限定为: 执行候选后产生的图差分只包含该候选声明的目标add/delete 编辑, 不引入候选之外的附加修改。

主指标包括closure accuracy、oracle selection accuracy 与strict event success。为刻画风险-覆盖关系, 另报告Coverage、Abstention、selected precision 和Wrong-selected rate, 其中

$$P_{\text{sel}} = \Pr(\text{Oracle} \mid \text{Select}).$$

4 方法

4.1 神经符号修复架构

本文将最终方法命名为ECR-REPAIR (Evidence-Constrained Semantic IR Repair)，并将其定位为证据约束的模块化神经符号本体修复框架。正文统一采用学术化模块名称；为保证与代码和实验日志可追溯，对应实现标识仅在此给出一次：Base Semantic IR Pipeline (M13)、Clause-Level Semantic Recovery (M14)、Temporal Anchor Recovery (M15)、Constraint-Aware Candidate Ranking and Selective Gate (V4.3) 以及Fallback Candidate Entailment Verifier (M16)。从功能上，框架包含四个角色：神经语义解释层 $\mathcal{N}_{\text{sem}}$ 、符号语义决策层 $\mathcal{S}_{\text{decision}}$ 、按需触发的神经候选验证层 $\mathcal{N}_{\text{verify}}$ ，以及形式符号闭环层 $\mathcal{S}_{\text{formal}}$ 。

$$\mathcal{N}_{\text{sem}} \rightarrow \mathcal{S}_{\text{decision}} \rightarrow [\mathcal{N}_{\text{verify}}] \rightarrow \mathcal{S}_{\text{formal}}.$$

$\mathcal{N}_{\text{sem}}$ 由Predicate-aware Evidence Focus、Base Semantic IR Pipeline 与必要的Clause-Level Semantic Recovery 构成，负责神经语义解释； $\mathcal{S}_{\text{decision}}$ 由Typed Semantic IR、Temporal Anchor Recovery 与Constraint-Aware Candidate Ranking and Selective Gate 构成，负责确定性/规则化的符号语义决策； $\mathcal{N}_{\text{verify}}$ 仅在排序歧义时调用Fallback Candidate Entailment Verifier； $\mathcal{S}_{\text{formal}}$ 则通过OWL/RDF Patch、Reasoner、RDF Graph Delta 和frozen repair/CQ regression checks 执行形式符号验证。该结构明确区分“神经解释”“符号约束”“神经备用验证”和“形式证明式检查”的责任边界。

排序阶段首先定义最大化候选集合

$$\mathcal{M}_E = \arg \max_{c_i \in C_E} S(c_i \mid \text{IR}, F; \tau_E),$$

其中 τ_E 表示事件的semantic type。若 $\mathcal{M}_E$ 只含一个候选且该候选满足冻结门控条件，则

$$c_{\text{rank}} = \begin{cases} c, & \mathcal{M}_E = \{c\} \text{ and } c \text{ is admissible,} \\ \perp, & \text{otherwise.} \end{cases}$$

若排序阶段不能给出唯一可接受候选，则Fallback Candidate Entailment Verifier 对候选逐个进行evidence entailment 验证：

$$c_{\text{verify}} = \begin{cases} c_j, & \text{if exactly one } c_j \in C_E \text{ is supported} \\ & \text{with high/medium confidence,} \\ \perp, & \text{otherwise.} \end{cases}$$

最终候选在统一闭环前定义为

$$\tilde{c}_E = \begin{cases} c_{\text{rank}}, & c_{\text{rank}} \neq \perp, \\ c_{\text{verify}}, & c_{\text{rank}} = \perp \wedge c_{\text{verify}} \neq \perp, \\ \text{ABSTAIN}, & \text{otherwise,} \end{cases}$$

并且

$$\hat{c}_E = \begin{cases} \tilde{c}_E, & \tilde{c}_E \neq \text{ABSTAIN} \wedge \text{OWLClosure}(\tilde{c}_E) = \text{true}, \\ \text{ABSTAIN}, & \text{otherwise.} \end{cases}$$

4.2 候选内容隔离的神经语义解释与Semantic IR

Base Semantic IR Pipeline 是神经语义解释层的主干，而不是一次“直接选答案”的提示。其处理链为：**predicate-aware evidence focus** $\rightarrow$ **task formulation** $\rightarrow$ **atomic fact extraction** $\rightarrow$ **evidence-grounded rule refinement** $\rightarrow$ **robust IR normalization**。EvidenceFocus 依据公开的subject、predicate、case

context 在已提供evidence window 中保留与目标谓词最相关的句段；task formulation 根据公开semantic type 与predicate 选择Temporal-Version、General Rule-Exception 或Cross-Sentence Scope 的字段约束；atomic extraction 要求事实附带对应evidence span，而不是只生成无出处结论。随后LLM 对draft facts 做一次只依赖公开证据的evidence-grounded critique/refinement，将其整理为候选内容隔离的类型化SEMANTIC IR。神经模块在这里负责将自然语言证据压缩为可追溯语义，而不拥有最终候选决策权；其输出必须进入后续符号层才能成为本体修复。

Robust IR normalization 只处理结构层问题，例如字段别名、可规范化标量和schema 兼容形式；它不读取候选，也不利用候选值补全缺失语义。若类型所需关键槽位在证据中不能恢复，则标记为incomplete，转入Clause-Level Semantic Recovery/Temporal Anchor Recovery 或最终fail-closed。这样，Base Semantic IR Pipeline 的职责是尽可能稳定地回答“证据说了什么”，而不是“哪个候选更像答案”。

4.3 三类事件语义类型

Temporal-Version 表示版本、生效日期、替代关系或当前有效状态变化，其IR 包含subject、old value、new value、effective time、relation、current value 和evidence 等字段。

General Rule-Exception 表示一般规则被例外、条件或优先级覆盖，其IR 包含subject、general rule、exception condition、exception rule、priority、result 和evidence 等字段。

Cross-Sentence Scope 表示限定语或作用域跨句附着，其IR 包含subject、statement、qualifier、scope target、scope relation、result 和evidence 等字段。

4.4 候选内容隔离协议

语义抽取和恢复模块不得读取候选的value、operation、id 或score。top-k identity、private oracle 与人工formal policy 同样不可见。初始Semantic IR 在候选内容不可见的条件下形成。唯一允许从候选决策层返回语义恢复层的是布尔控制信号ranking_failed=true；它只表示“需要再恢复一次”，不传递候选的领先、落后或冲突信息。因此本文使用“候选内容隔离”而非绝对“候选完全无影响”来描述该接口。

4.5 Clause-Level Semantic Recovery

Clause-Level Semantic Recovery 面向两类事件：一是Base Semantic IR Pipeline 后仍存在结构缺失的General Rule-Exception / Cross-Sentence Scope；二是IR 虽可排序但Constraint-Aware Candidate Ranking and Selective Gate 无法唯一决策的General Rule-Exception / Cross-Sentence Scope。前者直接在候选内容不可见条件下恢复；后者只接收ranking_failed=true，不接收candidate id、value、score、top-k identity 或任何候选派生语义。Clause-Level Semantic Recovery 在General Rule-Exception 中将一般规则、例外触发条件、例外结论和优先级拆成独立clause；在Cross-Sentence Scope 中将限定语、被限定声明和作用域附着关系拆开。只有恢复后能重新形成完整IR 并通过后续门控时才继续，否则保持abstain。

4.6 Temporal Anchor Recovery

Temporal Anchor Recovery 针对早期开发阶段识别出的temporal anchor ambiguity。版本化规范中，发布日期、发布日期引用、生效日期、被替代版本、当前有效版本和被引用历史

ECR-Repair: Evidence-Constrained Neuro-Symbolic Ontology Repair

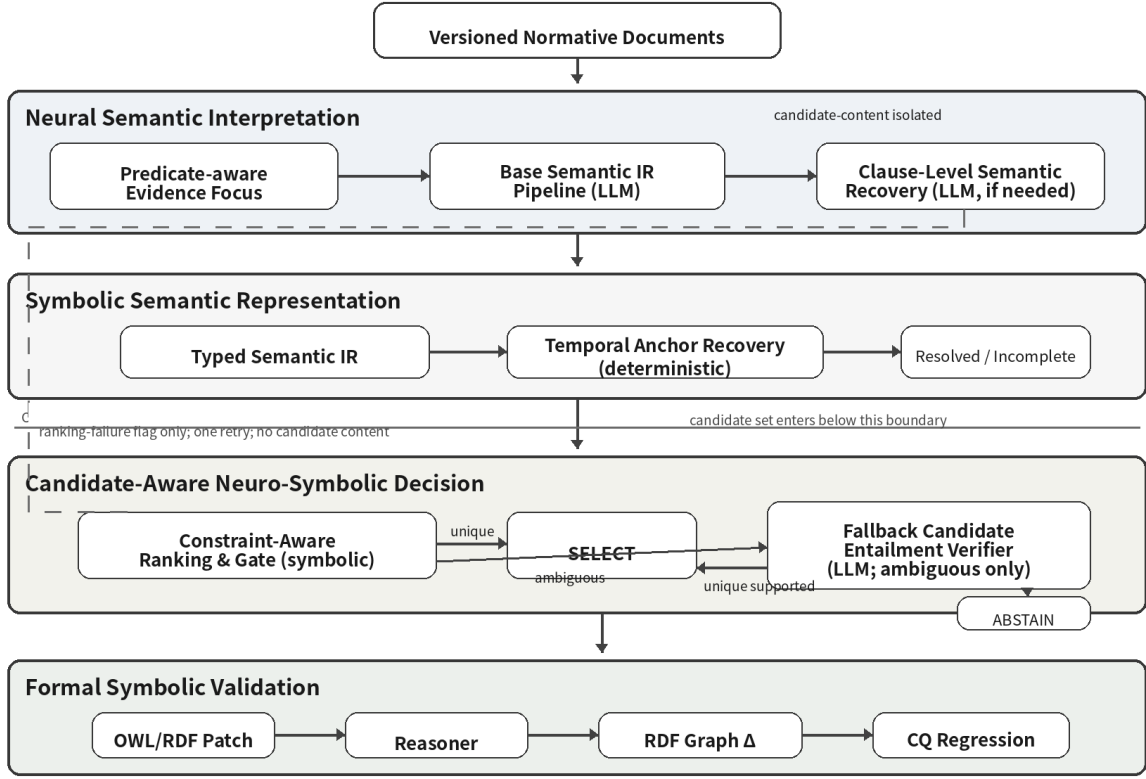


Figure 1: ECR-REPAIR的证据约束神经符号总体架构。**Semantic IR** 是neural-to-symbolic interface; 候选集合仅在**candidate-aware decision** 层可见。**Fallback Candidate Entailment Verifier** 只在符号排序存在歧义时触发, 虚线反馈只携带无语义的**ranking-failure flag**。

Table 1: Evidence-constrained ontology repair 算法

步骤	操作
1	$F \leftarrow \text{EvidenceFocus}(D_E, M_E)$, 只使用公开文档、事件元数据和证据窗口。
2	$IR \leftarrow \text{BaseIR}(F, M_E)$, 生成候选内容隔离的Semantic IR。
3	若General Rule-Exception / Cross-Sentence Scope 的IR 不完整, 则 $IR \leftarrow \text{ClauseRecovery}(IR, F, M_E)$; 若Temporal-Version 的时间锚点未解析, 则 $IR \leftarrow \text{TemporalRecovery}(IR, F, M_E)$ 。
4	在IR 固定后读取 C_E , 计算 $S_i = S(c_i IR, F; \tau_E)$, 得到 c_{rank} 或 $\text{ranking_failed}=\text{true}$ 。
5	若General Rule-Exception / Cross-Sentence Scope 仅返回 $\text{ranking_failed}=\text{true}$, Clause-Level Semantic Recovery 只接收该布尔信号做一次候选内容隔离recovery, 然后重新执行步骤4; retry 最多一次。
6	若排序仍不能给出唯一候选, 则调用Fallback Candidate Entailment Verifier, 得到 c_{verify} 或ABSTAIN。
7	对唯一候选执行OWL patch、reasoner、RDF graph delta 和frozen repair/CQ regression checks; 任一环节失败则返回ABSTAIN。
8	返回通过闭环验证的 $\hat{c}_E$ 。

版本可能同时出现; 直接把“最近日期”或“最后出现的版本号”当作current value 会产生系统性错误。Temporal Anchor Recovery 不读取任何候选, 也不使用候选score/margin。它把公开证据和文档时间元数据规范化为角色化anchor, 再由后

续Constraint-Aware Candidate Ranking and Selective Gate 将已固定的temporal IR 映射到候选。

Temporal Anchor Recovery 不是单标签分类器, 而是确定性的角色-关系规范化。令事件中出现的版本/规范实体集合

Table 2: Temporal Anchor Recovery 的候选内容隔离temporal anchor 规则

证据/元数据模式	规范化角色	处理原则
明确“obsoletes/ supersedes”关系	role: superseded/ current; relation: supersedes	被替代对象标记历史角色；替代关系的方向单独记录，不把角色压成单标签。
明确“updates/updated by”关系	relation: updates/ updatedBy	记录更新方向，不把update 自动等价于complete replacement；是否current 由正文证据另行决定。
明确effective date / takes effect	role: effective	将生效角色绑定到相应规范实体；publication 仅保留为独立角色，不覆盖effective。
正文明确current/this specification + 历史引用	role: current/reference	同一实体允许拥有多个角色；当前陈述与历史引用分开记录。
明确publication date 多重日期或版本证据 冲突	role: publication resolution: unresolved	作为文档时间元数据保留，不单独推出current。 不根据位置或“最新数字”猜测；在无法由公开文本关系消解时fail-closed。

为 $\mathcal{V}_E$ ，则

$$\text{NormalizeTemporal}(\text{Evidence}_E, \text{Metadata}_E) = (\mathcal{R}_E, \mathcal{T}_E, u_E),$$

其中

$$\mathcal{R}_E \subseteq \mathcal{V}_E \times \{\text{current, effective, superseded, reference, publication}\},$$

$$\mathcal{T}_E \subseteq \mathcal{V}_E \times \{\text{supersedes, updates, updatedBy}\} \times \mathcal{V}_E,$$

$$u_E \in \{\text{resolved, unresolved}\}.$$

因此同一实体可以同时具有current 与effective 等多个角色；版本之间的supersedes/updates 方向独立记录，而不是混进角色标签。仅当 $u_E = \text{unresolved}$ 且公开证据不能进一步消解时，Temporal Anchor Recovery 才fail-closed。表 2 中的规则处理的是角色、关系与方向，而不是候选分数。Constraint-Aware Candidate Ranking and Selective Gate 的min_score、min_margin 和temporal unique-top1 属于后续候选排序/门控，不属于Temporal Anchor Recovery。

示意例。若公开证据同时出现“RFC X published in year t_1 ”、“RFC Y obsoletes RFC X”以及“this specification takes effect at t_2 ”，Temporal Anchor Recovery 可得到下列角色与关系：

$$\mathcal{R}_E \supseteq \{(X, \text{publication}), (X, \text{superseded}), \\ (Y, \text{current}), (Y, \text{effective})\},$$

$$(Y, \text{supersedes}, X) \in \mathcal{T}_E.$$

这一示例仅用于说明算法，不属于v5 benchmark 的额外样本，也不参与结果统计。候选集合直到该temporal IR 固定后才进入Constraint-Aware Candidate Ranking and Selective Gate。

4.7 Constraint-Aware Candidate Ranking and Selective Gate

Constraint-Aware Candidate Ranking and Selective Gate 是从已固定SEMANTIC IR到有限候选集合的确定性映射层。为避免把实现写成不可审计的单一相似度，scorer 为每个候选保留一组可检查子分量：

$$\phi(c_i, \text{IR}, F) = (s_{\text{tok}}, s_{\text{exact}}, \\ s_{\text{family}}, s_{\text{code}}, s_{\text{num}}, s_{\text{sem}}, p_{\text{neg}}).$$

前五个分量分别刻画token overlap、exact match、family match、code match 与numeric/date match。 s_{sem} 表示类型相关的semantic-consistency bonus， p_{neg} 表示否定、方向或语义冲突penalty。最终分数写为

$$S(c_i | \text{IR}, F; \tau_E) = \text{Agg}(\phi(c_i, \text{IR}, F), \tau_E),$$

其中Agg 是冻结实现中的确定性聚合函数， τ_E 表示事件的semantic type 约束。本文不在v5 上重新估计任何scorer 参数；代码级聚合逻辑与阈值在方法freeze 中固定。Robust IR 只对IR 的字段/别名和可规范化结构做兼容处理，不使用候选内容“补答案”。

门控阈值在方法冻结时固定。默认最小分数为 $s_{\min} = 0.30$ ；General Rule-Exception 与Cross-Sentence Scope 使用 $s_{\min} = 0.24$ 。最小分数间隔固定为 $m_{\min} = 0.00$ 。

Temporal-Version 另启用temporal unique-top1。仅当角色/关系方向唯一、无显式冲突且不存在并列候选时，top-1 才可继续；否则返回ranking_failed=true。该信号最多触发一次Clause-Level Semantic Recovery，且不暴露候选分量、身份或领先信息。

选择性门控。键 门控的目标不是尽可能多地修复，而是在证据、Semantic IR 或候选决策不确定时拒绝修复。若IR 不完整、JSON 无效、候选得分低于冻结阈值、候选无法唯一判别，或后续Fallback Candidate Entailment Verifier 不能找到唯一supported 候选，系统均fail-closed。门控只决定候选是否具备进入下一阶段的资格，不把形式一致性等同于语义正确性。

4.8 Fallback Candidate Entailment Verifier

Fallback Candidate Entailment Verifier 仅在前序Constraint-Aware Candidate Ranking and Selective Gate 无法唯一决策时触发。v5 主实验使用DeepSeek API 的deepseek-chat: temperature=0，单次输出上限500 tokens，timeout=180s。API 不提供严格seed 控制，因此seed 只作为审计字段记录。

Verifier 对每个候选独立调用，而不是把三个候选一次性输入进行比较。每次调用可见当前候选的临时id、display value 和operation，其中id 只用于输出对齐。模型将候选判定为supported、contradicted、unrelated 或underspecified，并给出high/medium/low categorical confidence；该confidence 不是校准概率，也不解释为posterior probability。

只有当且仅当一个候选被判定为supported 且confidence 为high/medium 时, 候选才进入closure; 其余情况均触发abstain。deepseek-chat 仅作为服务端模型标识记录。由于云端服务可能更新, 精确复现还需保存运行日期、raw outputs 与配置/冻结manifest。

4.9 Formal OWL/RDF Closure and Repair/CQ Regression

每个候选操作统一视为有限的RDF/OWL 声明编辑集

$$\Delta_c = (\Delta_c^-, \Delta_c^+),$$

其中 Δ_c^- 表示候选声明的待删除集合, Δ_c^+ 表示待新增集合; 两者的元素均为目标RDF/OWL triple/statement。

对执行候选 c 后得到的本体 O_c , 实际图差分定义为

$$\Delta_{\text{graph}}(O_0, O_c) = (O_0 \setminus O_c, O_c \setminus O_0).$$

形式闭环要求

$$\Delta_{\text{graph}}(O_0, O_c) = \Delta_c.$$

若出现候选未声明的额外变化, 或声明编辑未按预期发生, 则closure 失败。该约束用于防止patch 的副作用, 而不是求解所有可能修复中的全局最小edit distance。

本文将最终回归接口统一称为**frozen repair/CQ regression checks**。当前冻结实现以event-specific triple-level repair checks 为主, 并保留competency-question-compatible 的回归接口; 这些检查与repair event 共享构造协议, 正式运行时不会根据模型输出动态生成或调整。因此它们用于验证已选patch 的声明图效应、逻辑一致性与预定义回归行为, 但不被视为独立**external semantic gold**, 也不承担语义oracle 的角色。形式闭环由OWL/RDF materialization、reasoner consistency、RDF graph-delta verification 与frozen repair/CQ regression checks 构成, 任一环节失败即返回ABSTAIN。其经验能力边界在RQ3 的stress test 中单独评价。

5 Benchmark 与评价协议

5.1 Benchmark 数据集

最终主实验使用external-real-holdout-v5-blind-large。该benchmark 在完整方法冻结后重新构建, 基于公开RFC 规范文档, 包含264 个经人工复核的repair event, 每个事件运行5 次, 共1320 次调用。数据覆盖11 个domain、13 个实际入选RFC source family 和三类semantic type; 候选来源注册表包含69 条公开来源记录。此前的v1/v3/v4 数据集用于诊断、开发、校准或补充验证, 不作为最终主结果数据集。

5.2 方法冻结与盲测协议

方法冻结manifest 的SHA-256 为e27619e2...5def068, 记录固定模块、阈值、输入隔离规则和禁止调参策略。随后构建v5, 完成264 个事件的oracle/support 复核、candidate notes 清洗、结构校验、深度质量审计和正式freeze。candidate notes 清洗只修改notes, 不修改candidate id、display value、operation 或oracle; 冻结方法不读取notes。本文所称**blind** 的操作含义是: 完整方法先于v5 构建被冻结、private oracle 与推理输入隔离、v5 结果出现后不再进行post-hoc 方法调参。该词不表示benchmark 由外部第三方双盲标注。

Table 3: 最终blind-large benchmark 概况

属性	数值
事件数	264
每事件运行次数	5
总调用次数	1320
Domain 数	11
实际入选source family 数	13
候选来源注册表规模	69
Semantic type 数	3
候选行数	792
清洗后candidate-note 泄漏行数	0
Benchmark manifest SHA-256	40fcc2c3...9bb0e038

5.3 候选构造与Identifier Regularity

每个v5 事件固定包含三个候选, 并保证其中恰有一个人工复核的oracle candidate。Oracle candidate 由公开evidence 与源OWL 中待修复断言之间的语义差异生成, 表示当前规范证据支持的修复。一个干扰候选通常保留旧版本、过期值或未修复语义; 另一个干扰候选是语义近邻但不被当前evidence 最好支持的可执行修复。所有候选都要求operation 可执行、OWL patch 可物化、display value 不含显式答案标记, candidate note 不被方法读取并经过泄漏审计。候选**identifier** 构造规律: 冻结v5 的构造模板将oracle 统一编码为CAND_002 (264/264), CAND_001 与CAND_003 为干扰候选。这一规律来自benchmark construction, 而不是private oracle 被暴露给推理流程, 但它形成潜在identifier shortcut, 因此本文单独实施per-event candidate-ID permutation audit。由于正确修复被保证包含在有限候选中, 本文不把结果解释为open-ended repair generation performance。

5.4 评价范围与实验单位

本文的实验单位是**event-level OWL/RDF patch**, 而不是以“本体总三元组数”作为自变量的scalability benchmark。候选operation 被归一化为有限的add/delete 编辑集, 并围绕目标subject/predicate 断言及其直接语义上下文执行。本文因此验证的是: 在已知潜在漂移事件条件下, 方法能否从规范证据选择并执行正确的局部修复。本文没有把大型共享本体的规模、TBox 深度或全库推理耗时作为实验因素, 也不据此主张对超大规模本体的推理扩展性。

5.5 比较方法与Baselines

采用阶段式消融: Base Semantic IR Pipeline; Base + Clause Recovery; Base + Clause + Temporal Recovery; 以及ECR-REPAIR (full Fallback Candidate Entailment Verifier configuration)。另外使用Direct LLM (Candidate Value) 和Direct LLM (Value + Formal Operation) 作为候选选择baseline。

5.6 评价指标与统计方法

由于同一事件的5 次运行共享同一事件级语义结构, 本文不把1320 个attempt 当作1320 个独立样本。Attempt-level accuracy 作为描述性运行指标报告, 其不确定性使用**event-clustered bootstrap**: 以264 个事件为抽样单位, 每次重采样事件时连

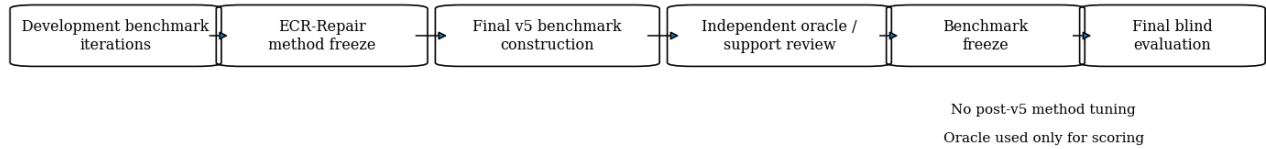


Figure 2: 方法冻结、benchmark 构建与最终盲测时间线。

同该事件的5个runs一起带入，采用10,000次percentile bootstrap。Strict event success本身以事件为单位，因此使用Wilson 95%区间。阶段差值同样使用event-clustered paired bootstrap；正文不再使用attempt-level McNemar p 值作为主要或补充推断，以避免伪重复带来的过度显著性。

5.7 实现与复现设置

实现与计算公平性说明。Direct LLM (Candidate Value) 与Direct LLM (Value + Formal Operation) 用于回答“直接暴露候选信息能做到多好”这一effectiveness baseline问题，并不是与ECR-REPAIR严格compute-matched的系统：ECR-REPAIR可能经历Clause-Level Semantic Recovery retry或Fallback Candidate Entailment Verifier per-candidate calls，而简单baseline的调用结构更短。因此本文不利用这些baseline声称单位token/单位延迟效率更高。云端deepseek-chat不能靠模型别名保证未来权重完全不变，复现时应同时保存调用日期、raw response与配置manifest。当前论文把OWL closure固定为项目中的materialization-reasoner-graph-delta-repair/CQ regression执行接口，但没有在现有实验记录中把具体reasoner/RDF library/repair-check harness的软件版本作为研究变量；为避免虚构未核验版本号，正式artifact release应随环境文件公开这些实现元数据。

6 评价结果

6.1 RQ1: ECR-Repair的总体修复效果如何？

主结果。主结果仍采用frozen v5、DeepSeek后端和冻结的完整方法。1320个attempts来自264个event的5次运行；attempt-level比例仅作描述，event-level不确定性按第5.6节的event-clustered protocol处理。唯一失败为General Rule-Exception事件H5_E017的一次abstain；最终wrong-selected patch为0。

直接候选基线。简单candidate-value baseline已达到93.48%，因此本文不把v5描述为“候选天然难以区分”的任务。Direct LLM (Candidate Value)的oracle-selection failure rate为86/1320 (6.52%)；ECR-REPAIR为1/1320 (0.08%)，且唯一一次失败为abstain而非错误选择。因此本文关注的是高基线下残余错误压缩与fail-closed风险控制。

统计不确定性与选择性修复。

6.2 RQ2: 哪些模块贡献了效果，Temporal Recovery是否改善了目标语义？

阶段式消融。Temporal Anchor Recovery是最大阶段增益来源。由于最终候选准确率不能直接证明temporal representation本身正确，本文进一步使用人工temporal gold对Temporal Anchor Recovery进行语义审计。

Temporal Anchor Recovery人工语义审计。88个Temporal-Version event使用人工填写的event-level temporal gold（不读取private oracle、候选答案、M13/M15输出），与正式v5的5次运行配对，共形成440个descriptive attempts。表11显示，Temporal Anchor Recovery对task-relevant anchor normalization的改善集中在role与effective-anchor层面，但并非对全部temporal relation extraction指标都提升。

在440 attempts中，有160次在role exact-frame上由M15改善；event-clustered mean从0.023提升到0.386。另一方面，relation type accuracy下降18.0 pp，说明M15更适合解释为面向修复决策的temporal anchor normalization，而不是完整temporal information extraction模块。特别是，最终Temporal-Version修复达到100% strict，并不意味着系统完整重建了全部时间关系；实验更支持“足够的结构化语义可支撑正确局部修复”这一较窄结论。

6.3 RQ3: 形式闭环能够提供什么安全保证？

乡乃乳兵乳乳乔乳。为区分“语义选择正确”与“形式补丁安全”，本文对每个v5 event生成5个确定性stress variants，共1320 cases。该实验不重复调用LLM，而直接测试frozen closure interface。

合法补丁接受率为100%。若把四类非-oracle stress variants作为整体，closure对其中三类结构/逻辑/回归破坏全部检出，但对264个语义错误而形式自治的patch完全无检测能力。因此closure的职责边界更准确地写为

$$\text{FormalClosureValid}(c) \Rightarrow \text{SemanticCorrect}(c).$$

也就是说，形式闭环通过并不蕴含候选在规范证据意义下语义正确。该结果支持把closure定位为semantic candidate selection之后的结构、逻辑、声明图效应和回归安全层，而不是独立语义oracle。

6.4 RQ4: 方法对候选规模、模型后端和候选Identifier是否稳健？

候选池规模鲁棒性。为检验主benchmark的三候选设定是否过度有利，本文从frozen v5派生 $k=5$ 与 $k=10$ hard-negative variants；parent event、evidence、oracle semantics和冻结方法保持不变，不重新调阈值、prompt或gate。每组均运行264 events $\times$ 5 runs。

候选池从3扩至10后，strict event success仅下降3个event（约1.1 pp），Temporal-Version始终88/88；主要损

Table 4: ECR-REPAIRv5 主实验实现配置

阶段	后端或执行方式	主要参数	候选内容可见性
Base Semantic IR Pipeline	DeepSeek API deepseek-chat	temp=0, max tokens=900, timeout=180s	不可见
Clause-Level Semantic Recovery	DeepSeek API deepseek-chat	two JSON calls, 700 tokens each, timeout=180s; retry 最多一次	不可见; 只可接收布尔失败信号
Temporal Anchor Recovery	Deterministic recovery	temporal role normalization; no candidate scores	不可见
Constraint-Aware Ranking/Gate	Constraint-aware scorer	min score=0.30; General Rule-Exception / Cross-Sentence Scope min score=0.24; min margin=0.00; temporal unique-top1; robust IR on	可见
Fallback Entailment Verifier	DeepSeek API deepseek-chat	per-candidate, temp=0, max tokens=500, timeout=180s	可见
OWL closure	RDF/OWL materialization + reasoner + repair/CQ checks	graph delta, consistency, frozen repair/CQ regression	仅可见已选候选

Table 5: ECR-REPAIR在最终frozen v5 blind-large benchmark 上的主结果

语义类型	Closure	Oracle	Strict event
ALL	1319/1320 (99.92%)	1319/1320 (99.92%)	263/264 (99.62%)
Temporal-Version	440/440 (100.00%)	440/440 (100.00%)	88/88 (100.00%)
General Rule-Exception	439/440 (99.77%)	439/440 (99.77%)	87/88 (98.86%)
Cross-Sentence Scope	440/440 (100.00%)	440/440 (100.00%)	88/88 (100.00%)

Table 6: 全量候选选择baseline 与ECR-REPAIR对比

方法	Oracle selection	OWL closure	备注
Direct LLM (Candidate Value)	1234/1320 (93.48%)	-	evidence + candidate value
Direct LLM (Value + Formal Operation)	1171/1320 (88.71%)	-	evidence + formal operation
Base Semantic IR Pipeline	1022/1320 (77.42%)	1022/1320 (77.42%)	基础Semantic IR 修复管道
ECR-REPAIR	1319/1320 (99.92%)	1319/1320 (99.92%)	完整闭环修复

Table 7: 阶段增益的event-clustered paired bootstrap

对比	平均提升	Event-clustered 95% CI
ECR-REPAIRvs Base Semantic IR Pipeline	22.50 pp	17.73–27.42 pp
Base + Clause Recovery vs Base Semantic IR Pipeline	2.73 pp	1.06–4.70 pp
Base + Clause + Temporal Recovery vs Base + Clause Recovery	18.71 pp	14.32–23.41 pp
ECR-REPAIRvs Base + Clause + Temporal Recovery	1.06 pp	0.00–2.42 pp

失集中在General Rule-Exception。该结果支持“对有限hard-negative candidate pool 扩容保持高稳定性”，但不外推到开放式候选生成或任意规模检索。

跨后端鲁棒性。为隔离LLM backend 影响，本文在同一frozen v5 ($k = 3$) 上仅把DeepSeek 替换为本

地Ollama qwen3.5:9b，其他方法逻辑、M15、ranking thresholds、candidate set 和closure 保持不变。每个后端均按 264×5 评价。

Qwen 与DeepSeek 相差1 个strict event。Qwen 运行中出现15/1320 (1.1%) 次Base IR timeout，集中在少数event，均按失败计入且未选择性补跑。因而该实验支持backend robustness/portability，而不声称模型无关性。

Table 8: 主结果的不确定性区间与独立单位

指标	点估计	95% 区间	独立/重采样单位
Attempt closure	1319/1320 (99.92%)	99.77–100.00%	264 events (每次携带5 runs)
Strict event success	263/264 (99.62%)	97.89–99.93%	264 events (Wilson)

Table 9: Coverage、abstention 与 selected precision

方法	Coverage	Abstention	Pr(Oracle Select)	Wrong selected
Base Semantic IR Pipeline	1034/1320 (78.33%)	286/1320 (21.67%)	1022/1034 (98.84%)	12/1320 (0.91%)
ECR-REPAIR	1319/1320 (99.92%)	1/1320 (0.08%)	1319/1319 (100.00%)	0/1320 (0.00%)

Table 10: 阶段式消融实验

方法阶段	Closure	Strict event	相对Base 提升	阶段增益
Base Semantic IR Pipeline	1022/1320 (77.42%)	195/264 (73.86%)	0.00 pp	–
Base + Clause Recovery	1058/1320 (80.15%)	205/264 (77.65%)	2.73 pp	2.73 pp
Base + Clause + Temporal Recovery	1305/1320 (98.86%)	261/264 (98.86%)	21.44 pp	18.71 pp
ECR-REPAIR	1319/1320 (99.92%)	263/264 (99.62%)	22.50 pp	1.06 pp

Table 11: 人工 temporal gold 上的Base IR 与 Temporal Anchor Recovery 对比

指标	Base IR	+ Temporal Anchor Recovery	Δ
Role exact-frame	2.3%	38.6%	+36.4 pp
Complete temporal frame	2.3%	38.6%	+36.4 pp
Effective anchor accuracy	8.0%	100.0%	+92.0 pp
Current anchor accuracy	100.0%	100.0%	0.0 pp
Role micro-F1	0.682	0.829	+0.147
Role macro-F1	0.628	0.700	+0.072
Relation type accuracy	62.3%	44.3%	-18.0 pp

元兮兴入光入兮兴入 全兴兵兴 允兵兴。冻结v5 的构造模板使264/264 个oracle 都使用CAND_002。因此早期global rename 与CSV row-order shuffle 并不能充分排除固定identifier shortcut: 全局rename 后oracle 仍统一映射为同一标签, row shuffle 又可能在canonical sorting 后恢复固定顺序。为直接检验这一风险, 本文在方法完全冻结的条件下, 对per-event identifier permutation variant 进行DeepSeek 264 × 5 的全量在线重跑。

seed=20260902 下, 仅在event 内重新分配candidate identifier; display value、operation 与语义均保持不变。Oracle ID 的事件级分布为

$$(n_{001}, n_{002}, n_{003}) = (71, 88, 105).$$

因此, 若系统始终选择字面CAND_002, 其准确率仅为88/264 = 33.33%。

全量在线结果为

$$\text{Closure} = \frac{1315}{1320} = 99.62\%, \quad \text{Strict} = \frac{262}{264} = 99.24\%.$$

oracle-correct 与closure 一致, 为1315/1320 (99.62%)。同时 $N_{\text{wrong}} = 0$, $N_{\text{abstain}} = 5$ 。成功选择的identifier 分布为

$$(\hat{n}_{001}, \hat{n}_{002}, \hat{n}_{003}) = (355, 436, 524),$$

说明选择随候选内容分散到三个identifier, 而非集中于CAND_002。5 次abstain 的oracle-ID 分布为(0, 4, 1)。其中H5_E003 的runs 2–5 为IR_FAIL_CLOSED, H5_E017 的run 2 为IR_RANK_TIE_ABSTAIN; M14 与M16 均无API error 或timeout。

event-level strict 配对结果为: 262 个event 在两种设置下均成功, H5_E003 仅original 成功, H5_E017 在两种设置下均strict-fail, 没有仅permutation 成功的event。以264 events 为聚类单位的10,000 次paired bootstrap 显示, Original–Permutation 的closure 差值为+0.30 pp (95% CI: 0.00–0.91 pp), strict-event 差值为+0.38 pp (95% CI: 0.00–1.14 pp)。在线permutation 仅带来很小的性能下降, literal CAND_002/fixed-identifier shortcut 无法解释观察到的99%+ 性能。

6.5 RQ5: Benchmark 的标注与provenance 边界是什么?

Benchmark 标注与provenance 审计。Paper-final validation 对benchmark 可靠性做了两类额外检查。第一, 分层抽取90 events (3 类各30) 后, 由一名人工reviewer A 与一个不读取private oracle、模型输出或A 结论的deterministic

Table 12: Formal closure stress test (264 events $\times$ 5 variants)

变体	含义	检出/接受结果	主要阶段
VALID_ORACLE_CONTROL	正确oracle patch	264/264 接受	final closure pass
EXTRA_UNDECLARED_EDIT	多余未声明图编辑	264/264 拒绝	graph delta
INCONSISTENT_PATCH	逻辑不一致patch	264/264 拒绝	reasoner
REPAIR_CHECK_BREAK	冻结repair regression 被破坏	264/264 拒绝	repair check
WRONG_SEMANTIC_PATCH	非oracle、但triple/逻辑自治	0/264 被拒绝	264/264 通过closure

Table 13: Candidate-pool size robustness

Pool	Closure	Strict events	Temporal	Rule-Exception	Cross-Sentence
$k = 3$	99.92%	263/264 (99.62%)	88/88	87/88	88/88
$k = 5$	99.39%	261/264 (98.86%)	88/88	86/88	87/88
$k = 10$	99.09%	260/264 (98.48%)	88/88	85/88	87/88

Table 14: 同一v5 benchmark 上的backend robustness

Backend	Closure	Strict events	Temporal	Rule-Exception	Cross-Sentence
DeepSeek	99.92%	263/264 (99.62%)	88/88	87/88	88/88
Qwen3.5:9b	99.24%	262/264 (99.24%)	88/88	88/88	86/88

Table 15: Candidate identifier sensitivity and shortcut audit

Setting	Oracle-ID distribution	Closure	Strict event	解释
Original	0/264/0	99.92%	263/264	identifier 恒定; 0 wrong, 1 abstain
Global rename	0/264/0 equivalent	99.47%	261/264	oracle identity 仍恒定; 0 wrong, 7 abstain
Per-event permutation (online)	71/88/105	99.62%	262/264	always-CAND_002=33.33%; 0 wrong, 5 abstain

unique-token heuristic B 独立产生诊断标签。该实验不是两名人工标注者的inter-annotator study, 因此不将oracle-choice agreement 解释为人工一致性证据。Semantic type 的A-B agreement 为93.3%, Cohen’s $\kappa = 0.90$, 6 个分歧进入adjudication; evidence-sufficiency agreement 为100%。Oracle-choice 虽为90/90, 但因frozen v5 oracle 全部为CAND_002、标签零方差, Cohen’s κ 不可定义, 该结果只作consistency diagnostic。

第二, 本文按更严格的provenance-complete drift 条件筛选: 要求显式lineage, 并满足

$$\text{Support}(a_0, D_{t_0}) = \text{supported},$$

$$\text{Support}(a_0, D_{t_1}) \neq \text{supported},$$

$$\text{Support}(a^*, D_{t_1}) = \text{supported}.$$

subset selection 不读取模型prediction。基于当前冻结public RFC cache 和严格人工review, 最终入选event 数为 $N = 0$ 。主要原因包括部分被替代历史RFC 不在本地cache, 以及现有evidence window 无法在不放宽标准的情况下同时证明完整 t_0/t_1 support chain。因此这不是性能为0, 而是没有满足严格admission criteria 的可评价样本。该负结果要求本文将

主benchmark 明确解释为evidence-relative repair benchmark, 而非provenance-complete historical drift benchmark。

6.6 失败模式与测试切片分析

领域与来源族切片。11 个领域中有10 个达到100% closure, 唯一非满分领域为dns_security, 119/120 (99.17%); 13 个实际入选RFC 来源族中有12 个达到100%, 唯一非满分来源族为IETF RFC 4035, 同为119/120。这里报告的是frozen-method domain/source-family test-slice stability, 不是训练式leave-one-domain-out, 也不将其表述为严格cross-domain transfer/generalization experiment。

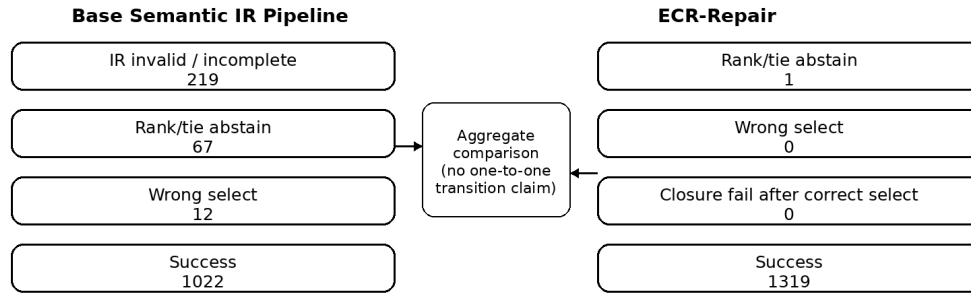
失败案例与失败链路。最终v5 主实验只剩1 次失败, 来自H5_E017 第5 次运行, 属于General Rule-Exception; Fallback Candidate Entailment Verifier 未找到唯一supported 候选, 因此abstain。最终没有产生wrong-selected patch, 也没有出现correct selection 后的OWL closure failure。

7 讨论

实验表明, 本任务的关键并不是简单保证LLM 输出JSON 格式正确, 而是建立稳定、可诊断的neural-to-symbolic semantic

Table 16: Base Semantic IR Pipeline 与 ECR-REPAIR 的失败状态边际分布

失败或成功状态	Base Semantic IR Pipeline	ECR-REPAIR
IR invalid / incomplete	219	0
Rank/tie abstain	67	1
Wrong select	12	0
OWL closure fail after correct select	0	0
Success	1022	1319

Failure-State Marginals: Base vs. ECR-Repair

Counts are marginal state totals; differences are not interpreted as paired causal recovery transitions.

Figure 3: Base Semantic IR Pipeline 与 ECR-REPAIR 的失败状态压缩概览。该图用于展示边际失败量级变化，不将总量差直接解释为逐样本因果转移。

interface。ECR-REPAIR的神经模块负责从自然语言规范中解释证据与处理剩余语义歧义，符号模块则把这些解释约束为Typed Semantic IR、确定性时间关系、候选可接受性条件和OWL/RDF可执行修改。因此，最终修复并不是某一次LLM生成的直接产物，而是神经语义解释与符号约束/验证交替协作的结果。candidate-content isolation 进一步保证候选语义不会反向污染前序神经解释。

新增validation 使几个模块的责任边界更加清晰。首先，Temporal Anchor Recovery 的人工gold 审计说明其最大价值来自effective/current 等task-relevant anchor roles，而不是完整关系图恢复：effective-anchor 从8% 升至100%，但relation typing 反而下降。这意味着最终repair decision 所需的是sufficient structured semantics，而不是对所有temporal facts 的exhaustive reconstruction。其次，formal closure stress test 证明closure 能稳定拦截未声明图编辑、逻辑不一致和冻结repair-check regression violation，却无法识别“语义错但形式自治”的patch。因此semantic correctness 必须由evidence-grounded selection 层承担，形式闭环不能替代语义oracle。

候选池和backend 实验也缩小了“99.92% 只来自单一设置”的解释空间。候选数从3 增至10 后strict event success 仍为260/264，下降约1.1 pp，且Temporal-Version 始终满分；在同一v5 上只把LLM 后端替换为本地Qwen3.5:9b，strict event success 为262/264，与DeepSeek 仅差1 个event。因而当前证据支持有限候选池robustness 与backend portability，但仍不足以声称open-ended generation、任意规模候选池或model-independence。

候选identifier 审计暴露并处理了一个重要benchmark regularity: v5 构造模板把所有oracle 固定为CAND_002。全局rename

与row-order shuffle 并不能充分排除这一shortcut，因为oracle identity 仍全局恒定。更强的per-event permutation 将oracle ID 打散为71/88/105 后，冻结M13-M16 pipeline 的DeepSeek 264x5 全量在线重跑仍达到99.62% closure、262/264 strict 和0 wrong-select; literal CAND_002 strategy 仅为33.33%。Original 与permutation 的paired event-clustered bootstrap 差值也很小 (closure +0.30 pp, 95% CI 0.00–0.91; strict +0.38 pp, 95% CI 0.00–1.14)。这表明观察到的99%+ 性能不能由literal CAND_002/fixed-identifier shortcut 解释；但该实验只排除这一特定shortcut，不意味着benchmark 不存在其他潜在构造规律。

最后，strict provenance audit 的 $N = 0$ 是对论文问题边界的重要约束，而不是可忽略的空结果。它说明当前v5 虽基于真实RFC evidence 构造并支持evidence-relative repair decision，却不能在现有冻结evidence cache 下对每个事件恢复provenance-complete $t_0 \rightarrow t_1$ historical drift chain。因此本文保留“semantic drift”作为问题背景和evidence-relative 操作性概念，但不把99.92% 解读为历史版本漂移发现或完整provenance reconstruction 的成功率。

8 有效性威胁

来源领域限制。最终benchmark 主要来自RFC/IETF 风格的公开技术规范。虽然覆盖11 个domain 和13 个来源族，但不能直接推出对法律、合同、临床指南等文本同样有效。

数据构造与identifier 规律。v5 使用真实RFC 文档和evidence window，但repair event、subject/predicate metadata 与候选集合仍由实验协议构造；冻结模板还把oracle 统一编码

为CAND_002。后者形成潜在shortcut，已通过per-event identifier permutation 的DeepSeek 264x5 全量在线重跑主动审计；在oracle ID 打散为71/88/105 后仍取得99.62% closure、262/264 strict 和0 wrong-select，因此literal fixed-identifier 策略不能解释主结果，但benchmark construction regularity 本身仍应透明披露。

标注可靠性边界。当前oracle/support 主要由单人复核。90-event validation 中第二路B 是不读取oracle/模型/A 标签的deterministic heuristic，而非第二名独立人工标注者，因此semantic-type $\kappa = 0.90$ 只表示human-heuristic consistency，不能当作human inter-annotator agreement。更强的benchmark reliability 仍需要真正独立的第二人工标注。

语义漂移provenance。严格要求显式lineage、 t_0 支持旧断言以及 t_1 否定旧断言并支持新断言的provenance audit 得到 $N = 0$ 。因此主实验只支持evidence-relative semantic drift repair，不支持provenance-complete historical drift reconstruction。该限制同时受到当前public RFC cache 不完整的影响。

条件式任务边界。v5 假设潜在事件已定位、相关evidence window 已提供、有限候选中包含正确修复。本文不评价open-corpus retrieval recall、candidate generation recall 或drift discovery precision/recall/F1。虽然新增 $k = 5/10$ hard-negative variants 表明有限候选池扩容下仍高稳定，但不能外推到开放式生成或任意大候选空间。

形式化闭环边界。closure stress test 显示结构性非法patch（额外图编辑、reasoner inconsistency、repair-check break）可被全部拦截，但264/264 个语义错误而形式自洽的patch 仍通过closure。因此closure 只能提供structural/logical/regression safety，不能代替evidence-grounded semantic selection。

CQ/repair-check 独立性。当前冻结回归层主要是event-specific triple-level repair checks，与repair event 共享构造协议，不能被视作独立external semantic gold。未来需要由独立任务作者构建更丰富的independent CQ/SPARQL suite。

Temporal Recovery 的目标范围。人工gold 上M15 对effective anchor 与role frame 显著改善，但relation-type accuracy 下降。本文因此只把它定位为task-oriented temporal anchor normalization，而非通用、完整的temporal relation extraction 系统。

本体规模与推理扩展性。当前评价单位是event-level local patch，而不是以全本体大小、TBox 深度或大规模ABox 为变量的scalability study。因此结果不能直接外推到超大本体的全局推理成本。

候选内容隔离边界。General Rule-Exception / Cross-Sentence Scope 路径允许一个ranking_failed=true 布尔信号触发一次recovery，因此候选集合会影响“是否重试”，但具体候选内容不会反馈到语义层。本文明确把该机制称为candidate-content isolation，而非绝对candidate independence。

统计独立性。同一事件的5 次run 不是独立样本，因此attempt-level 比例只作描述，主结果区间和阶段差值以event-clustered bootstrap 为主要推断；strict event 以事件为单位使用Wilson 区间。新增robustness 表中的百分比在未给出clustered CI 时只作描述性比较。

模型后端与超时。DeepSeek 是可变的云服务标识，未来更新可能影响精确复现；Qwen3.5:9b v5 run 出现15/1320 Base IR timeouts，均按失败计且未补跑。两后端高接近结果支持portability，但不支持model-independence。

Candidate-ID permutation 的解释边界。全量在线permutation test 直接排除了“始终选择字面CAND_002 / 固定identifier”这一具体shortcut 对99%+ 结果的解释：打散identifier 后closure 仍为99.62%，且wrong-select 为0。该实验并不能证明benchmark 完全不存在其他shortcut，也不改变v5 候选构造本身是人工协议设计这一事实；因此本文只将其解释为针对literal candidate-identifier bias 的反事实稳健性证据。

计算量与baseline 公平性。简单候选baseline 用于effectiveness 对照，并非与ECR-REPAIRcompute-matched；后者可能产生recovery/verifier 额外调用。因此本文不从当前表格推断成本或延迟优势。

实现元数据。当前正文固定了materialization-reasoner-graph-delta-repair-check 的逻辑接口，但现有稿件没有核验并列出具体的reasoner/RDF library/harness 的软件版本。正式artifact 应公布环境/依赖信息；在此之前本文不主张跨reasoner 或跨library 的完全复现等价性。

最终盲测边界。v1/v3/v4 属于诊断、校准或补充验证数据；v5 是完整方法冻结后重新构建并freeze 的主盲测。paper-final validation 只做冻结方法的额外审计与robustness test，不据结果重新调整M13-M16、ranking thresholds 或prompts。

9 结论

本文提出ECR-REPAIR，一种面向版本化规范文档的证据约束模块化神经符号本体修复框架。方法以Semantic IR 作为neural-to-symbolic interface：LLM 负责候选内容隔离的证据语义解释与按需候选蕴含验证，确定性恢复与约束感知排序负责符号语义决策，OWL/RDF materialization、Reasoner、图差分与冻结repair/CQ regression checks 负责形式闭环。在“潜在事件已定位、相关evidence 已提供且正确修复位于有限候选中”的主v5 条件式benchmark 上，264 个RFC-based repair events、1320 次调用得到99.92% closure accuracy、263/264（99.62%）strict event success 和0 wrong-selected patch。

Paper-final validation 进一步使该结论从“高准确率”转向“高准确率及其可审计边界”：人工temporal gold 支持Temporal Anchor Recovery 对effective anchor 的显著改善，但不支持其作为完整temporal relation extractor；closure stress test 证明形式闭环能发现结构/逻辑/回归破坏，却不能识别语义错误而自洽的patch；候选池扩至 $k = 10$ 后strict 仍为260/264，本地Qwen3.5:9b 后端为262/264；per-event candidate-ID permutation 的DeepSeek 全量在线重跑在打破CAND_002 固定identifier 后仍达到1315/1320（99.62%）closure、262/264（99.24%）strict 和0 wrong-select，而always-CAND_002 baseline 仅为33.33%。与此同时，strict historical provenance audit 得到 $N = 0$ ，因此本文只主张evidence-relative semantic drift repair，而不主张provenance-complete historical drift reconstruction。

总体上，本文的核心观点是：高可靠本体修复不能仅依赖更强的语言模型，也不能把形式一致性误当成语义正确性；更可行的路径是通过显式Semantic IR、候选内容隔离、选择性abstention、符号约束与形式安全检查构建责任边界清晰的神经-符号接口。当前结论仍不覆盖drift discovery、开放语料retrieval、open-ended candidate generation、任意大候选空间或超大本体推理扩展性。

生成式AI 使用说明

OpenAI ChatGPT 被用于协助论文的语言编辑、结构整理、学术化命名、LaTeX 排版与表达改进。作者对研究问题、实验设

计、数据、分析、结论及最终文本进行了审阅和编辑，并对论文全部内容的真实性与正确性承担责任。

References

- [1] Shqiponja Ahmetaj, Robert David, Axel Polleres, and Mantas Simkus. 2022. Repairing SHACL Constraint Violations Using Answer Set Programming. In *The Semantic Web – ISWC 2022 (Lecture Notes in Computer Science, Vol. 13489)*. Springer, 375–391.
- [2] Shqiponja Ahmetaj, Robert David, Axel Polleres, and Mantas Šimkus. 2025. A Logic Programming Approach to Repairing SHACL Constraint Violations. *Transactions on Graph Data and Knowledge* 3, 3 (2025).
- [3] Giorgos Flouris, Dimitris Manakanatas, Haridimos Kondylakis, Dimitris Plexousakis, and Grigoris Antoniou. 2008. Ontology Change: Classification and Survey. *The Knowledge Engineering Review* 23, 2 (2008), 117–152.
- [4] Yonatan Geifman and Ran El-Yaniv. 2017. Selective Classification for Deep Neural Networks. In *Advances in Neural Information Processing Systems*.
- [5] Keerat Guliani, Deepkamal Gill, David Landsman, Nima Eshraghi, Krishna Kumar, and Lovedeep Gondara. 2026. De Jure: Iterative LLM Self-Refinement for Structured Extraction of Regulatory Rules. Publication metadata to be verified before final submission.
- [6] Michael Hartung, James F. Terwilliger, and Erhard Rahm. 2013. Ontology Evolution: A Process-Centric Survey. *The Knowledge Engineering Review* 28, 1 (2013), 45–75.
- [7] Ernesto Jimenez-Ruiz, Bernardo Cuenca Grau, Yujiao Zhou, and Ian Horrocks. 2015. Ontology Alignment Repair through Modularization and Confidence-Based Heuristics. *PLOS ONE* 10, 12 (2015), e0144807.
- [8] Aditya Kalyanpur, Bijan Parsia, Evren Sirin, and Bernardo Cuenca-Grau. 2006. Repairing Unsatisfiable Concepts in OWL Ontologies. In *The Semantic Web: Research and Applications (Lecture Notes in Computer Science, Vol. 4011)*. Springer, 170–184.
- [9] Holger Knublauch and Dimitris Kontokostas. 2017. Shapes Constraint Language (SHACL). W3C Recommendation.
- [10] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*.
- [11] Jiayi Li, Daniel Garijo, and María Poveda-Villalón. 2026. Large Language Models for Ontology Engineering: A Systematic Literature Review. *Semantic Web* (2026). Bibliographic details to be verified before final submission.
- [12] Anna Sofia Lippolis, Mohammad Javad Saeedizade, Robin Keskisärkkä, Sara Zuppiroli, Miguel Ceriani, Aldo Gangemi, Eva Blomqvist, and Andrea Giovanni Nuzzolese. 2025. Ontology Generation Using Large Language Models. Publication metadata to be verified before final submission.
- [13] Damith Premasiri and Tharindu Ranasinghe. 2025. Survey on Legal Information Extraction: Current Status and Open Challenges. *Knowledge and Information Systems* (2025).
- [14] Anmol Singhal and Travis D. Breaux. 2025. Legal Requirements Translation from Law. In *33rd IEEE International Requirements Engineering Conference*. 205–217.
- [15] Jiashuo Sun, Pengcheng Jiang, Saizhuo Wang, Jiajun Fan, Heng Wang, Siru Ouyang, Ming Zhong, Yizhu Jiao, Chengsong Huang, Xueqiang Xu, Jiawei Han, et al. 2026. Rethinking the Reranker: Boundary-Aware Evidence Selection for Robust Retrieval-Augmented Generation. Publication metadata to be verified before final submission.
- [16] Sinh Trong Vu, Minh Le Nguyen, and Ken Satoh. 2022. Abstract Meaning Representation for Legal Documents: An Empirical Research on a Human-Annotated Dataset. *Artificial Intelligence and Law* 30, 2 (2022), 221–243.
- [17] W3C OWL Working Group. 2012. OWL 2 Web Ontology Language Document Overview. W3C Recommendation. <https://www.w3.org/TR/owl2-overview/>